

Electricity Demand Forecasting

A Comprehensive Machine Learning Project Report



Prepared by:
Sachethana Abeyrathne

June 14, 2026

Contents

1	Dataset Description	2
2	Project Objective	2
3	Data Processing	2
3.1	Basic Information and Indexing	2
3.2	Handling Missing Values	3
4	Feature Engineering	4
4.1	Lagged and Rolling Features	4
5	Exploratory Data Analysis (EDA)	4
5.1	Raw Demand Over Time	4
5.2	Demand by Hour of Day	5
5.3	Demand by Month	5
5.4	Demand vs. Temperature	6
5.5	Correlation Matrix	7
6	Model Development	7
6.1	XGBoost Algorithm	7
7	Model Evaluation	8
7.1	Visualizing Predictions	8
8	Deployment: Streamlit Application	9
8.1	Application Mechanics and Utility	9
9	Conclusion	10

1 Dataset Description

The dataset utilized in this project records historical electricity demand alongside corresponding weather conditions. It is a time-series dataset where observations are recorded chronologically. Key variables include the **Timestamp**, meteorological features like **Temperature** and **Humidity**, and the target variable, **Demand** (measured in Megawatts).

	hour	dayofweek	month	year	dayofyear	quarter	Temperature	Humidity	Demand
Timestamp									
2020-01-01	0.0	2.0	1.0	2020.0	1.0	1	3.000000	61.288951	2457.119872
2020-01-01	1.0	2.0	1.0	2020.0	1.0	1	3.000000	52.873702	2269.904712
2020-01-01	2.0	2.0	1.0	2020.0	1.0	1	4.244482	36.341783	2215.640403
2020-01-01	3.0	2.0	1.0	2020.0	1.0	1	3.000000	72.629378	2174.232413
2020-01-01	4.0	2.0	1.0	2020.0	1.0	1	3.881208	90.582444	2472.453006
...	...	...	...	...	...	...	...	...	...
2024-12-31	19.0	1.0	12.0	2024.0	366.0	4	3.956838	43.287161	4689.693109
2024-12-31	20.0	1.0	12.0	2024.0	366.0	4	3.118824	51.705756	4331.249224
2024-12-31	21.0	1.0	12.0	2024.0	366.0	4	3.000000	40.565916	4015.979957
2024-12-31	22.0	1.0	12.0	2024.0	366.0	4	3.000000	51.998107	3353.241682
2024-12-31	23.0	1.0	12.0	2024.0	366.0	4	6.037472	59.931925	3219.023339

43844 rows x 9 columns

Figure 1: Snapshot of the raw Electricity Demand Dataset.

2 Project Objective

The primary task of this project is to develop a robust machine learning pipeline capable of forecasting future electricity demand. By learning from historical consumption patterns, seasonal trends, and weather influences, the model aims to predict hourly and daily electricity requirements. Accurate forecasting is critical for grid stability, optimal resource allocation, and preventing power outages or energy wastage.

3 Data Processing

Before feeding data into the machine learning model, it must be cleaned and structured. The following steps were executed:

3.1 Basic Information and Indexing

Initial inspection was conducted to understand data types and dimensions. The **Timestamp** column was converted to a **datetime** format and set as the DataFrame's index. This is a crucial step in time-series analysis as it allows for time-based slicing, resampling, and plotting.

```

... <class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 43844 entries, 2020-01-01 to 2024-12-31
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   hour        43844 non-null  float64
1   dayofweek    43844 non-null  float64
2   month        43844 non-null  float64
3   year         43844 non-null  float64
4   dayofyear    43844 non-null  float64
5   quarter      43844 non-null  int32
6   Temperature  43844 non-null  float64
7   Humidity     43844 non-null  float64
8   Demand       43844 non-null  float64
dtypes: float64(8), int32(1)
memory usage: 3.2 MB

```

Figure 2: Basic information and schema of the dataset.

	Temperature	Humidity	Demand
count	43841.000000	43838.000000	43841.000000
mean	25.067788	59.903007	5000.790976
std	12.821725	18.342604	1412.527409
min	3.000000	20.000000	1611.954020
25%	15.210186	46.241224	4015.668472
50%	25.003212	59.986720	5013.053367
75%	34.740971	73.796820	6000.803082
max	50.000000	95.000000	11910.705100

Figure 3: Summary statistics of the numerical features.

3.2 Handling Missing Values

Rows containing entirely missing values were dropped. For partial missing data, the following specific imputation techniques were applied to maintain the temporal integrity of the dataset:

- **Forward Fill (ffill):** Applied to temporal features (hour, dayofweek, etc.). This technique propagates the last known valid observation forward to fill the gap. It is highly effective for time-series data where the state remains constant until a new observation is recorded.
- **Backward Fill (bfill):** Applied to Temperature and Humidity. This method uses the next valid observation to fill gaps moving backward. It is useful when weather data temporarily drops out but the surrounding conditions are stable.

- **Time Interpolation** (`interpolate(method='time')`): Applied to the `Demand` column. Instead of blindly filling with a previous or next value, time interpolation draws a linear line between the missing points based on the time distance. This provides a smooth, mathematically sound estimation of demand during the missing interval.

4 Feature Engineering

To capture the complexity of electricity consumption, new features were derived from the existing data. Temporal features such as `Quarter`, `Week_of_Year`, and an `is_weekend` indicator were created to help the model recognize seasonal and weekly behavioral shifts (e.g., lower demand on weekends).

4.1 Lagged and Rolling Features

Past demand values are exceptionally strong predictors of future demand. The following features were engineered:

- **Lag 24hr & Lag 168hr**: Captures the demand at the exact same hour the previous day (24h) and the previous week (168h). This explicitly teaches the model daily and weekly cyclical patterns.
- **Rolling Mean & Standard Deviation (24hr)**: These capture the smoothed trend and the volatility of demand over the last day, providing context about whether overall consumption is currently trending upward or downward.

5 Exploratory Data Analysis (EDA)

5.1 Raw Demand Over Time

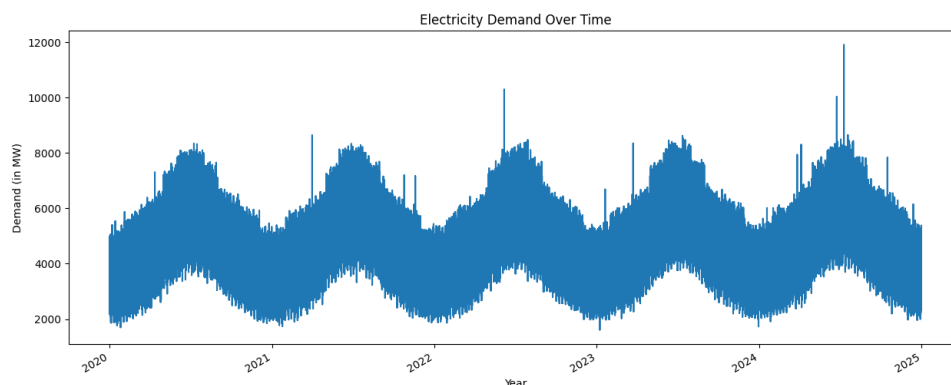


Figure 4: Raw Electricity Demand Over Time.

Interpretation: The plot displays the overarching macro-trend of demand. Clear periodic spikes and dips can be observed, indicating strong yearly seasonality, likely corresponding to extreme summer or winter months where heating or cooling is heavily utilized.

5.2 Demand by Hour of Day

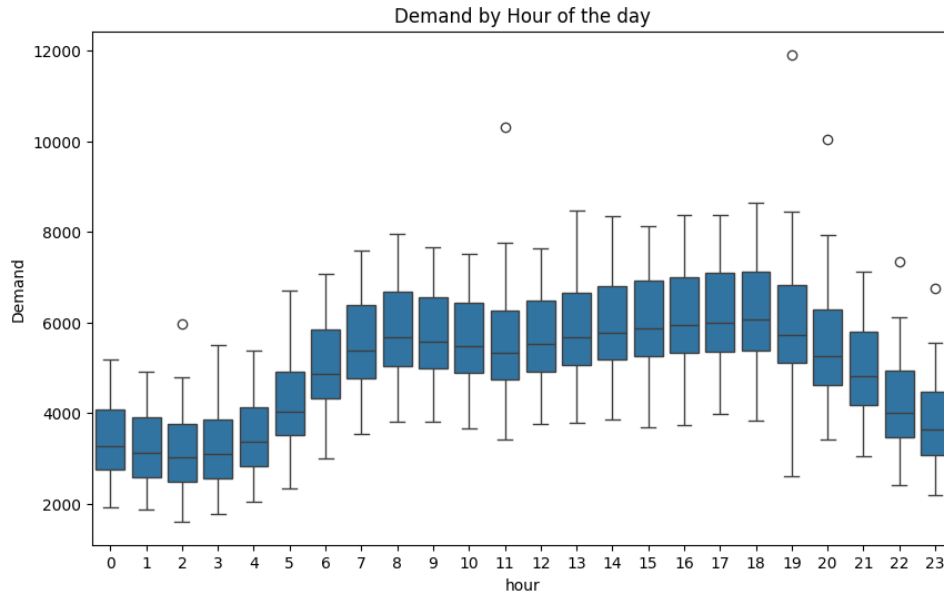


Figure 5: Boxplot of Demand by Hour of Day.

Interpretation: This visualizes intra-day seasonality. Demand typically drops during the early morning hours (2 AM - 5 AM), ramps up as businesses open, and peaks in the early evening (6 PM - 9 PM) when residents return home and turn on appliances.

5.3 Demand by Month

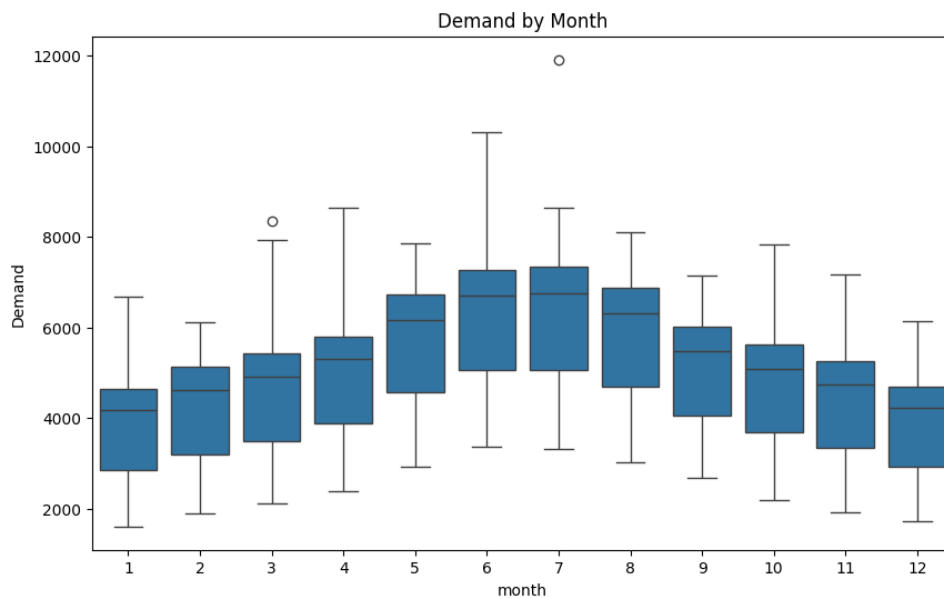


Figure 6: Boxplot of Demand by Month.

Interpretation: This highlights macro-seasonality. Certain months show significantly higher medians and wider interquartile ranges, representing periods of intense weather

requiring HVAC usage.

5.4 Demand vs. Temperature

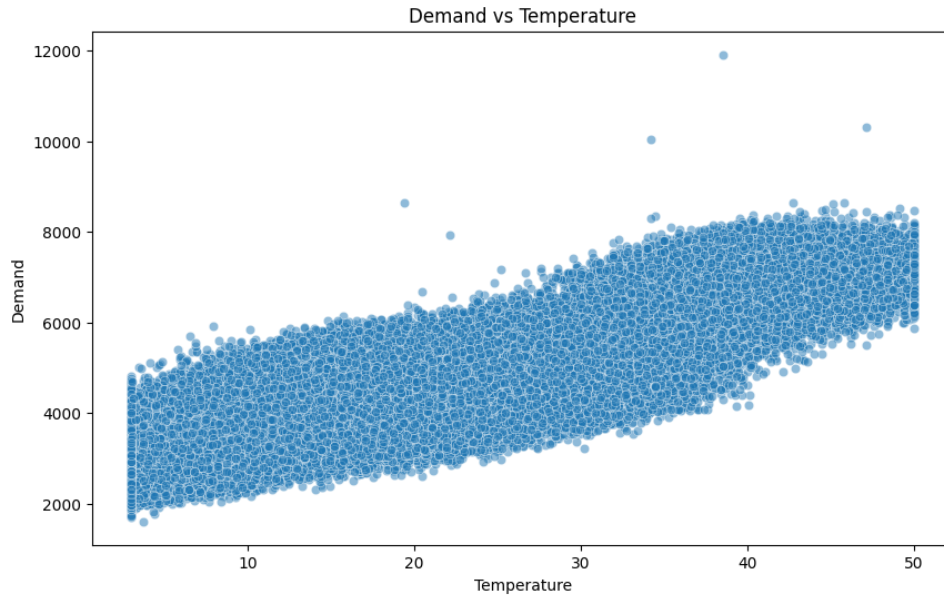


Figure 7: Scatter plot of Demand vs. Temperature.

Interpretation: Typically, this forms a U-shape or V-shape. Demand is high during very low temperatures (heating) and high during very high temperatures (air conditioning), dipping to its lowest during mild, comfortable weather.

5.5 Correlation Matrix

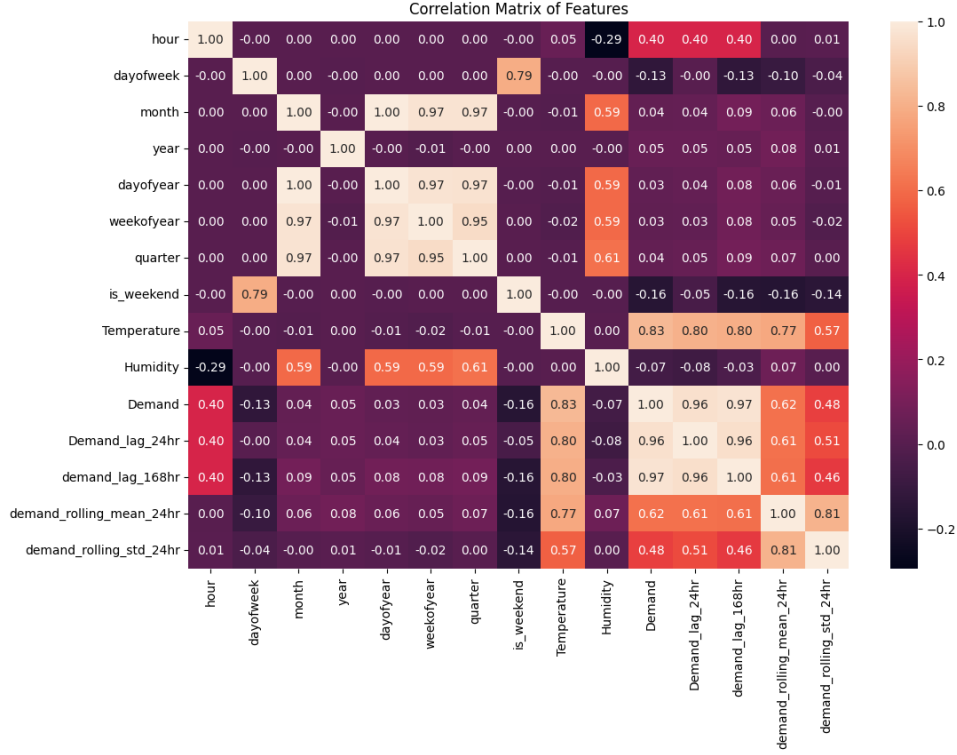


Figure 8: Heatmap of Feature Correlations.

Interpretation: The heatmap quantitatively confirms that lagged features (`Demand_lag_24hr`) have the highest positive correlation with the target demand. Temperature shows distinct correlations that validate our scatter plot findings.

6 Model Development

The dataset was split sequentially into 80% training data and 20% testing data to evaluate the model on unseen future periods.

6.1 XGBoost Algorithm

The primary model utilized was **XGBoost (Extreme Gradient Boosting)**. XGBoost is a powerful algorithm for time-series forecasting because it naturally handles non-linear relationships, seasonality, and trends.

Configuration Overview: The `XGBRegressor` was initialized to train for up to 1000 boosting rounds (decision trees). To prevent overfitting, early stopping was implemented—halting training if the model’s performance on a validation set failed to improve for 50 consecutive rounds. Furthermore, each tree’s contribution was scaled down by a conservative learning rate of 0.01. The objective function was set to minimize squared error, ensuring the model heavily penalizes large forecasting mistakes.

7 Model Evaluation

The model's performance on the test set yielded the following metrics:

- **RMSE (Root Mean Squared Error):** 175.22
- **MAE (Mean Absolute Error):** 123.47

Comment on Accuracy: An MAE of 123.47 indicates that, on average, the model's hourly predictions are off by roughly 123 Megawatts. Given that baseline electricity demand operates in the thousands of Megawatts, this represents an incredibly low percentage error (typically around 3-5%). The RMSE, being slightly higher, indicates that while there are occasional larger errors (perhaps during anomalous weather events or holidays), the overall fit is highly accurate and reliable for operational planning.

7.1 Visualizing Predictions

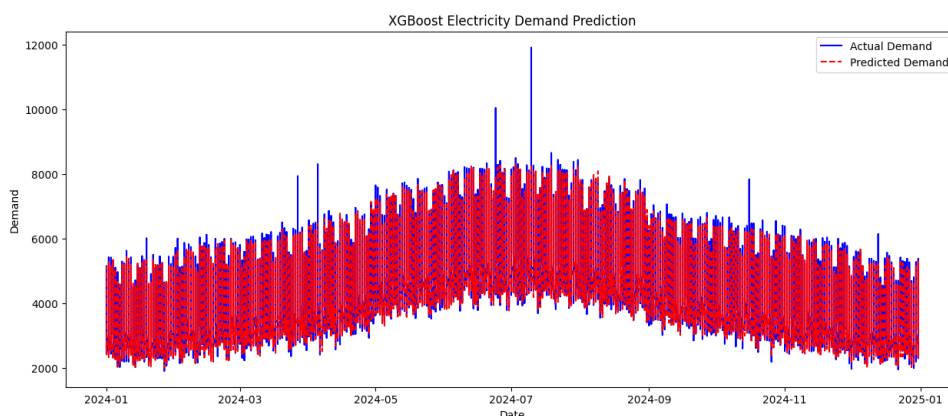


Figure 9: Actual vs. Predicted Electricity Demand.

Comment: The overlapping line plot shows that the XGBoost predictions tightly follow the actual demand curve, successfully capturing both the peaks and troughs of the daily cycles.

	Actual Demand	Predicted Demand
Timestamp		
2024-01-01	2720.606072	2660.955322
2024-01-01	2669.647167	2497.245361
2024-01-01	2435.578924	2400.469727
2024-01-01	2641.043699	2600.166992
2024-01-01	2687.949643	2575.836670
2024-01-01	3309.357599	3242.642822
2024-01-01	4084.103103	3847.981689
2024-01-01	4599.967024	4809.303711
2024-01-01	4889.096038	4798.926758
2024-01-01	4769.658664	4814.977539
2024-01-01	4782.431338	4941.937012
2024-01-01	4457.939794	4267.124512
2024-01-01	4873.890779	4887.100098
2024-01-01	4911.990265	4767.756348

Figure 10: Tabular view of Top 50 Actual vs Predicted values.

8 Deployment: Streamlit Application

To make the model accessible to stakeholders, an interactive web application was built using **Streamlit**.

8.1 Application Mechanics and Utility

The application loads the pre-trained XGBoost model and historical data. Users can specify a start date and the number of days they wish to forecast (up to 14 days). The app dynamically generates a future dataframe, computing expected temporal features (hour, day of week) and utilizing the last known actual values for temperature, humidity, and lagged metrics to bootstrap the prediction.

Why it is useful: Instead of requiring stakeholders to understand Python or execute code, this tool abstracts the complexity behind a clean User Interface. Grid operators can interactively simulate the upcoming week’s demand with the click of a button, viewing both hourly granular forecasts and daily aggregated totals to make immediate supply decisions.

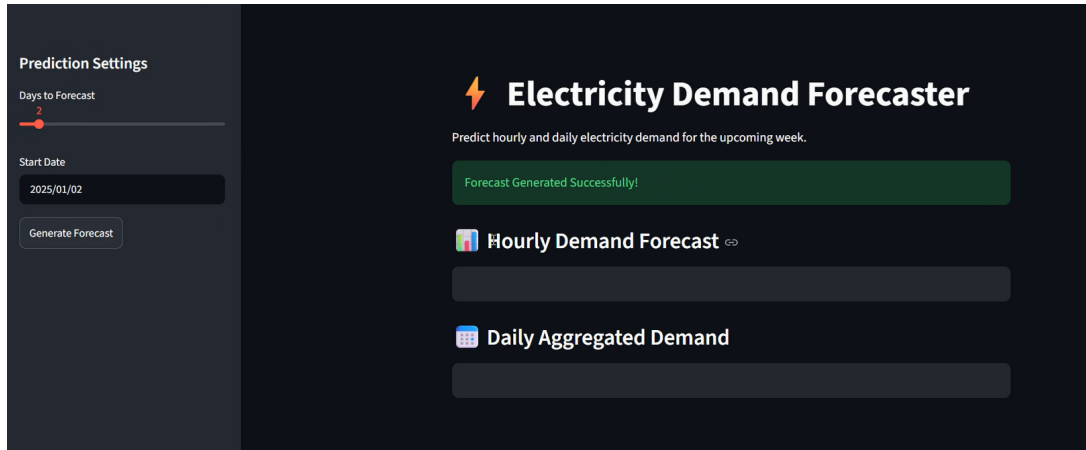


Figure 11: User Interface of the deployed Streamlit Forecasting Application.

9 Conclusion

This project successfully developed an end-to-end machine learning pipeline for electricity demand forecasting. By meticulously cleaning the data, engineering temporal and lagged features, and training an optimized XGBoost model, we achieved an average error of just 123 MW. The encapsulation of this complex model into a Streamlit application ensures that the predictive power is immediately usable for real-world grid management, resource optimization, and strategic energy planning.